

■ POKÉDEX MODULAR

Documentação Técnica Completa

Linha por linha · Separado por módulo

Gerado automaticamente a partir do código-fonte

Projeto: PokedexModular | Stack: HTML + CSS + JavaScript (ES Modules)

ÍNDICE

1. Visão Geral da Arquitetura

2. index.html — Estrutura HTML

3. js/state.js — Estado Global

- state (objeto principal)

4. js/config.js — Configurações e Constantes

- TOTAL_POKEMON
- TYPE_BACKGROUND
- OVERLAY_COLORS
- TYPE_COLORS
- TYPE_COLORS_LIGHT
- menuActions

5. js/utils.js — Utilitários

- capitalize

6. js/dom.js — Referências ao DOM

- \$ (atalho)
- Todas as variáveis de elementos

7. js/api.js — Comunicação com a PokeAPI

- fetchPokemon
- fetchSpecies
- loadBasicPokemonList

8. js/ui.js — Interface e Renderização

- renderList
- highlightSelected
- updateCounters
- updateSaveBtn
- updateTypeBackground
- updateTypeFilterColor

9. js/evolutions.js — Cadeia de Evoluções

- extractId
- loadEvolutions

10. js/rain.js — Animação de Chuva de Pokémon

- initPokemonRain
- stopRain
- resizeRainCanvas
- loadRainImages
- spawnFallingPokemon
- startRain
- animateRain

11. js/events.js — Eventos de Teclado e Clique

- initEvents

12. js/main.js — Orquestrador Principal

- simulateLoading
- snorlaxWakeUp
- showTitleScreen
- dropLogo
- showMenu
- updateMenuSelection
- goToPokedex
- openCredits
- closeCredits
- backToMenu
- initPokedex
- fetchAllPokemonTypes
- selectPokemon
- setupSearch
- setupTabs
- setupTypeFilter
- applyFilters
- toggleFavorite
- selectPokemonByListIndex
- scrollListToIndex
- executeMenuAction
- init

13. css/ — Estilos CSS (resumo por arquivo)

1. VISÃO GERAL DA ARQUITETURA

O PokedexModular é uma aplicação web de página única (SPA) que consome a **PokéAPI** para exibir informações dos 151 Pokémon originais. O código é organizado em módulos ES6, cada um com responsabilidade única:

Módulo	Responsabilidade
state.js	Estado reativo global compartilhado entre todos os módulos
config.js	Constantes: total de Pokémon, cores por tipo, backgrounds
utils.js	Funções auxiliares genéricas (capitalize)
dom.js	Centralizador de referências a elementos HTML (getElementById)
api.js	Fetch da PokéAPI com cache em memória
ui.js	Renderização da lista, contadores, botões, fundo por tipo
evolutions.js	Busca e exibe a cadeia de evoluções do Pokémon selecionado
rain.js	Animação de sprites caindo na tela de créditos (Canvas API)
events.js	Registra eventos de teclado e clique passando callbacks
main.js	Orquestrador: inicializa telas, controla fluxo, une todos os módulos

2. index.html — ESTRUTURA HTML

Arquivo único de marcação. Define as **4 telas** da aplicação como **divs** empilhadas (z-index), controladas por CSS (.hidden) e JavaScript:

#loading-screen

Tela de carregamento — exibe o Snorlax dormindo com a animação de 'Z z z' enquanto os dados são preparados.

#snorlax-container

Container do Snorlax; recebe a classe **waking-up** via JS para acionar a animação de acordar.

#snorlax-status

Elemento <p> que exibe os textos 'Z z z', 'Z z z .' etc e, ao acordar, o '!'. Animado por JS.

#title-screen

Tela de título (menu inicial). Começa hidden; exibida após o loading.

#forest-bg / #forest-img

Fundo animado GIF da floresta; recebe a classe **reveal** para surgir via fadeIn.

#logo-container / #logo-img

Imagem do título 'Pokédex'; recebe a classe **drop-in** para a animação de queda.

#menu-options

Contêiner dos itens de menu (START e Credits). Recebe a classe **show** para aparecer.

.menu-item[data-action]

Cada item de menu carrega um **data-action** ('start' ou 'credits') lido pelo JS para executar a ação correta.

#bottom-bar

Barra vermelha inferior com dica de navegação (↑↓ ENTER).

#credits-screen

Tela de créditos com fundo desfocado, canvas de chuva de Pokémon e cards dos integrantes.

#pokemon-rain-canvas

<canvas> usado pela rain.js para desenhar os sprites caindo.

#credits-light-spin

Efeito decorativo de raios de luz girando (CSS conic-gradient + animação lightSpin).

.member-card

Card de cada integrante com foto, nome, Instagram, e-mail e função no projeto.

#pokedex-screen

Tela principal da Pokédex. Dividida em header + pokedex-body (painel esquerdo + direito).

#type-background

Camadas absolutas de fundo por tipo: **#type-bg-image** (imagem blur) + **#type-bg-overlay** (cor semitransparente).

#pokedex-header

Barra vermelha superior com botão de voltar, título e contadores (Vistos / Salvos).

#panel-left

Painel esquerdo: nome, número, sprite, tipos, stats, info extra, evoluções e botão de favorito.

#pokemon-stats / .stat-row

Seis linhas de stat (HP, ATK, DEF, SP.A, SP.D, SPD) cada uma com label, barra animada e valor numérico.

#evolution-chain

Container onde evolutions.js injeta os itens clicáveis da cadeia evolutiva.

#save-btn

Botão 'Salvar Favorito / ★ Salvo!'; evento de clique registrado em main.js.

#panel-right

Painel direito: campo de busca, abas (Todos / Favoritos), filtro de tipo, lista de Pokémon.

#search-input

Input de texto para busca por nome ou número; ouvido pelo evento 'input' em main.js.

#tabs .tab

Botões de aba com atributo **data-tab** ('all' ou 'favorites'), alternados por JS.

#type-filter-select

<select> com todos os 18 tipos; ao mudar, filtra a lista pelo tipo selecionado.

#pokemon-list

Div que recebe os itens .poke-list-item gerados dinamicamente por ui.js.

3. js/state.js — ESTADO GLOBAL

Exporta um único objeto **state** importado por quase todos os demais módulos. Funciona como uma 'store' simples — não há reatividade automática; os módulos leem e escrevem diretamente nas propriedades.

Variáveis do objeto state

state.allPokemonBasic (Array)

Lista completa dos 151 Pokémon com formato {id, name}. Populada por `api.loadBasicPokemonList()`. Usada como fonte de dados para filtros e renderização da lista.

state.currentPokemonId (Number)

ID do Pokémon atualmente exibido no painel esquerdo. Começa em 1 (Bulbasaur). Atualizado sempre que o usuário clica em um item da lista ou navega pelo teclado.

state.currentTab (String)

Aba ativa na lista: 'all' (todos) ou 'favorites' (favoritos). Controla qual subconjunto de Pokémon é exibido.

state.currentTypeFilter (String)

Tipo selecionado no filtro de tipos (ex: 'fire', 'water'). String vazia significa sem filtro.

state.seenSet (Set)

Conjunto de IDs de Pokémon já visualizados na sessão atual. Usado para o contador 'Vistos' no header.

state.favorites (Array)

Lista de IDs marcados como favorito. Inicializada a partir do localStorage (chave 'pokedex-favorites') para persistir entre sessões.

state.pokemonCache (Object {id: data})

Cache em memória dos dados completos de Pokémon retornados pela PokéAPI. Evita requisições repetidas. Chave = ID numérico.

state.speciesCache (Object {id: data})

Cache em memória dos dados de espécie (/pokemon-species/:id). Contém a URL da cadeia evolutiva.

state.evoCache (Object {url: data})

Cache em memória das cadeias evolutivas. Chave = URL completa da API retornada por speciesCache.

state.pokemonTypeMap (Object {id: string[]})

Mapeia ID de Pokémon para array de tipos (ex: {4: ['fire']}). Populado incrementalmente por `fetchAllPokemonTypes` para colorir a lista.

state.menuIndex (Number)

Índice do item de menu selecionado no teclado (0 = START, 1 = Credits). Permite navegação com ↑/↓.

state.filteredList (Array)

Subconjunto de `allPokemonBasic` após aplicar os filtros ativos (aba + busca + tipo). É o array renderizado na lista.

state.listNavIndex (Number)

Índice do item atualmente destacado na `filteredList` durante navegação por teclado.

state.currentScreen (String)

Tela ativa: 'loading', 'title', 'credits', 'transitioning' ou 'pokedex'. Controla qual bloco de código de eventos está ativo.

state.pokedexInitialized (Boolean)

Garante que `initPokedex()` seja chamada uma única vez, evitando duplicação de dados ao voltar ao menu e retornar.

4. js/config.js — CONFIGURAÇÕES E CONSTANTES

Centraliza todos os dados estáticos da aplicação. Nenhuma lógica, apenas exportações de constantes utilizadas pelos demais módulos.

TOTAL_POKEMON (Number (151))

Quantidade total de Pokémon suportados pela aplicação. Usada em `api.loadBasicPokemonList()` como limite da requisição à PokéAPI e em `rain.js` para limitar o range de IDs aleatórios.

TYPE_BACKGROUNDS (Object {tipo: caminho})

Mapeia cada tipo de Pokémon para o caminho relativo do arquivo de imagem de fundo. Utilizado por `ui.updateTypeBackground()` para trocar o fundo do painel esquerdo conforme o tipo primário do Pokémon selecionado.

OVERLAY_COLORS (Object {tipo: rgba})

Mapeia cada tipo para uma cor semitransparente em formato rgba. Sobreposta à imagem de fundo (`type-bg-overlay`) para escurecer/tingir o fundo de acordo com o tipo, garantindo legibilidade do texto.

TYPE_COLORS (Object {tipo: hex})

Cor principal de cada tipo em hexadecimal (ex: fire → `#f08030`). Usada para colorir os badges de tipo (`.type-badge`), bordas esquerda dos itens da lista, e a bolinha pokébola miniatura.

TYPE_COLORS_LIGHT (Object {tipo: rgba})

Versão translúcida (35% opacidade) das cores por tipo. Usada como fundo do select de filtro de tipos (`ui.updateTypeFilterColor()`) quando um tipo específico está selecionado.

menuActions (Array ['start', 'credits'])

Define a ordem dos itens de menu. Usado em `events.js` para calcular o índice de navegação circular com `↑/↓` e em `main.js` para despachar a ação correta ao pressionar Enter.

5. js/utils.js — UTILITÁRIOS

Módulo mínimo com funções genéricas sem dependências externas.

capitalize (Arrow Function (s: string) → string)

Transforma o primeiro caractere da string em maiúsculo e mantém o restante inalterado. Usada em toda a aplicação para exibir nomes de Pokémon, habilidades e tipos formatados corretamente, pois a PokéAPI retorna todos os nomes em letras minúsculas.

s

String de entrada, geralmente o nome do Pokémon em lowercase (ex: 'bulbasaur').

Retorno: String com primeira letra maiúscula (ex: 'Bulbasaur').

```
export const capitalize = s => s.charAt(0).toUpperCase() + s.slice(1);
```

6. js/dom.js — REFERÊNCIAS AO DOM

Centraliza todos os `document.getElementById()` em um único lugar. Isso evita repetição e garante que, se um ID mudar no HTML, só este arquivo precisa ser atualizado. As variáveis são constantes exportadas.

\$ (Arrow Function (id: string) → HTMLElement)

Atalho para `document.getElementById(id)`. Definida localmente e também re-exportada para uso em `rain.js` e `main.js` (`dom.$('stat-hp')` etc).

id

String com o ID do elemento HTML.

Retorno: Referência ao elemento HTML correspondente.

Variáveis de elementos exportados

loadingScreen #loading-screen

Container da tela de carregamento (Snorlax).

snorlaxContainer #snorlax-container

Container do Snorlax; recebe classe 'waking-up'.

snorlaxStatus #snorlax-status

Parágrafo com 'Z z z' — atualizado pelo `simulateLoading`.

titleScreen #title-screen

Div da tela de título (menu inicial).

forestBg #forest-bg

Div do fundo floresta; recebe classe 'reveal'.

logoContainer #logo-container

Div do logo; recebe classe 'drop-in'.

menuOptions #menu-options

Container dos itens START e Credits.

creditsScreen #credits-screen

Tela de créditos completa.

pokedexScreen #pokedex-screen

Tela principal da Pokédex.

pokemonList #pokemon-list

Div onde os itens da lista são injetados.

searchInput #search-input

Input de busca por nome/número.

displayName #pokemon-display-name

Span do nome do Pokémon selecionado.

displayNumber #pokemon-display-number

Span do número (#001).

spriteImg #pokemon-sprite

Tag `<img>` do sprite/GIF do Pokémon.

typesContainer #pokemon-types

Div onde os badges de tipo são inseridos.

evoChain #evolution-chain

Container da cadeia de evoluções.

saveBtn #save-btn

Botão 'Salvar Favorito'.

seenCountEl #seen-count

 com contagem de Pokémon vistos.

savedCountEl #saved-count

 com contagem de favoritos.

pokemonWeight #pokemon-weight

Span do peso em kg.

pokemonHeight #pokemon-height

Span da altura em metros.

pokemonAbility #pokemon-ability

Span da habilidade principal.

typeBgImage #type-bg-image

Div do fundo desfocado (background-image por tipo).

typeBgOverlay #type-bg-overlay

Div da sobreposição colorida por tipo.

btnBackMenu #btn-back-menu

Botão '■ MENU'.

typeFilterSelect #type-filter-select

<select> de filtro por tipo.

7. js/api.js — COMUNICAÇÃO COM A POKEAPI

Todas as requisições de rede passam por este módulo. Implementa cache em memória via **state.pokemonCache**, **state.speciesCache** e **state.evoCache** para não repetir chamadas à API.

fetchPokemon (async Function (id: number) → Object | null)

Busca os dados completos de um Pokémon pelo ID. Antes de fazer a requisição, verifica se o resultado já está em `state.pokemonCache[id]`; se sim, retorna imediatamente. Em caso de erro de rede, retorna null.

id

Número inteiro de 1 a 151 representando o ID do Pokémon.

Retorno: Objeto JSON da PokéAPI com stats, types, abilities, sprites etc., ou null em caso de falha.

Obs: Os dados são armazenados em `state.pokemonCache[id]` após a primeira requisição bem-sucedida.

fetchSpecies (async Function (id: number) → Object | null)

Busca os dados de espécie de um Pokémon (`/pokemon-species/:id`). Contém informações como texto do Pokédex, taxa de captura, habitat e, crucialmente, a URL da cadeia evolutiva (`species.evolution_chain.url`) usada por `evolutions.js`.

id

ID numérico do Pokémon.

Retorno: Objeto JSON da PokéAPI com dados de espécie, incluindo `evolution_chain.url`.

Obs: Cache em `state.speciesCache[id]`.

loadBasicPokemonList (async Function () → void)

Faz uma única requisição buscando name e URL dos primeiros TOTAL_POKEMON Pokémon. Transforma o resultado em um array de objetos `{id, name}` e armazena em `state.allPokemonBasic`. Se a requisição falhar, gera um fallback com nomes genéricos 'pokemon-N'.

Retorno: void. Resultado armazenado em `state.allPokemonBasic`.

Obs: Chamada uma única vez em `initPokedex()`.

8. js/ui.js — INTERFACE E RENDERIZAÇÃO

Responsável por toda a manipulação de DOM relacionada à lista de Pokémon, contadores, botão de favorito e fundo dinâmico por tipo.

renderList (Function (list: Array, onSelect: Function) → void)

Limpa completamente #pokemon-list e reconstrói todos os itens com base no array recebido. Para cada Pokémon cria uma div .poke-list-item com: mini-pokébola colorida, número formatado (#001), nome capitalizado, pontos coloridos de tipo e estrela de favorito. Adiciona evento de clique que chama onSelect(p.id) e atualiza state.listNavIndex.

list

Array de objetos {id, name} a ser renderizado (pode ser filteredList ou subconjunto).

onSelect

Callback chamado com o ID do Pokémon quando o usuário clica no item.

Retorno: void. **Atualiza o DOM diretamente.**

Obs: A borda esquerda de cada item é colorida com a cor do tipo primário via TYPE_COLORS.

highlightSelected (Function () → void)

Percorre todos os .poke-list-item existentes, compara o número de cada um com state.currentPokemonId e alterna a classe 'selected'. Também sincroniza state.listNavIndex com o índice visual do item selecionado.

Retorno: void.

updateCounters (Function () → void)

Atualiza os textos dos contadores no header: seenCountEl com o tamanho de state.seenSet e savedCountEl com o tamanho de state.favorites.

Retorno: void.

updateSaveBtn (Function () → void)

Verifica se state.currentPokemonId está em state.favorites. Se sim, muda o texto do botão para '★ Salvo!' e adiciona a classe 'saved' (muda a cor para verde). Caso contrário, exibe '■ Salvar Favorito' (amarelo).

Retorno: void.

updateTypeBackground (Function (type: string) → void)

Atualiza o fundo dinâmico da tela da Pokédex conforme o tipo primário do Pokémon selecionado. Define a background-image de #type-bg-image com o arquivo de TYPE_BACKGROUND[type] e aplica a cor de OVERLAY_COLORS[type] em #type-bg-overlay.

type

String do tipo primário (ex: 'fire', 'water').

Retorno: void.

updateTypeFilterColor (Function () → void)

Lê o valor atual do #type-filter-select. Se um tipo estiver selecionado, altera o background, cor e borda do select para refletir a cor daquele tipo (usando TYPE_COLORS_LIGHT e TYPE_COLORS). Se 'Todos os Tipos' estiver selecionado, restaura o estilo padrão branco.

Retorno: void.

9. js/evolutions.js — CADEIA DE EVOLUÇÕES

Busca e renderiza a cadeia evolutiva do Pokémon selecionado usando a PokeAPI evolution-chain endpoint.

extractId (Function (url: string) → number [privada])

Função auxiliar interna (não exportada). Recebe uma URL da PokéAPI (ex:

https://pokeapi.co/api/v2/pokemon-species/4/) e extrai o ID numérico do último segmento do path, após remover a barra final.

url

String de URL completa da PokéAPI.

Retorno: Número inteiro representando o ID.

loadEvolutions (async Function (id: number, onSelect: Function) → void)

Orquestra a busca e renderização da cadeia evolutiva: 1. Exibe 'Carregando...' em #evolution-chain. 2. Chama fetchSpecies(id) para obter a URL da evolution_chain. 3. Verifica cache em state.evoCache[url]; se não houver, faz fetch da URL. 4. Percorre a cadeia (chain → evolves_to[0]) montando um array [{name, id}]. 5. Para cada evo, cria um .evo-item com imagem sprite e nome; destaca o atual com a classe 'current'; adiciona setas '→' entre itens. 6. Clique em qualquer evo chama onSelect(evo.id), navegando para aquele Pokémon.

id

ID do Pokémon selecionado atualmente.

onSelect

Callback (geralmente main.selectPokemon) chamado ao clicar em uma evolução.

Retorno: void. Atualiza o DOM de #evolution-chain.

Obs: Suporta apenas cadeias lineares (sempre pega evolves_to[0]). Ramificações (Eevee, Slowpoke) mostram apenas o primeiro galho.

10. js/rain.js — ANIMAÇÃO DE CHUVA DE POKÉMON

Cria uma animação de sprites de Pokémon caindo na tela de créditos usando a **Canvas API**. Os sprites são GIFs animados carregados via `Image()`.

Variáveis de módulo (privadas)

rainCanvas (`HTMLCanvasElement` | `undefined`)

Referência ao elemento `<canvas id='pokemon-rain-canvas'>`. Inicializada em `initPokemonRain()`.

rainCtx (`CanvasRenderingContext2D` | `undefined`)

Contexto 2D do canvas, obtido via `rainCanvas.getContext('2d')`. Usado para todas as operações de desenho.

rainAnimId (`number` | `null`)

ID retornado por `requestAnimationFrame(animateRain)`. Armazenado para poder cancelar a animação com `cancelAnimationFrame` em `stopRain()`.

fallingPokemons (`Array`)

Array dos objetos de sprites atualmente em queda. Cada objeto contém: `img`, `x`, `y`, `size`, `speed`, `opacity`, `wobbleAmp`, `wobbleSpeed`, `wobbleOffset`, `rotation`, `rotSpeed`.

pokemonRainImages (`Array`)

Pool de 30 imagens carregadas aleatoriamente. `startRain()` sorteia desse pool para criar novos sprites.

rainImagesLoaded (`Boolean`)

Flag que evita recarregar as imagens se a tela de créditos for aberta mais de uma vez na mesma sessão.

Funções exportadas

initPokemonRain (`Function ()` → `void` [exportada])

Ponto de entrada da chuva. Obtém o canvas e o contexto, ajusta o tamanho ao viewport, registra listener de `resize` e decide se carrega as imagens (primeira vez) ou inicia a animação diretamente (imagens já no pool).

stopRain (`Function ()` → `void` [exportada])

Para a animação cancelando o `requestAnimationFrame`, limpa `fallingPokemons` e apaga o canvas com `clearRect`. Chamada ao fechar a tela de créditos.

Funções privadas

resizeRainCanvas (`Function ()` → `void`)

Ajusta `rainCanvas.width` e `rainCanvas.height` para `window.innerWidth` / `window.innerHeight`. Chamada no `init` e em cada evento de `resize` da janela.

loadRainImages (`Function ()` → `void`)

Gera 30 IDs únicos aleatórios (1-151), cria um `HTMLImageElement` para cada GIF animado na PokeAPI (generation-v black-white animated). Usa um contador 'loaded'; quando todos terminam (com sucesso ou erro), define `rainImagesLoaded = true` e chama `startRain()`.

spawnFallingPokemon (`Function ()` → `Object` | `null`)

Cria um novo objeto de sprite com propriedades aleatórias: imagem sorteada do pool, posição X aleatória, Y negativo (acima da tela), tamanho entre 70–120px, velocidade 0.4–1.0, opacidade 0.2–0.5, parâmetros de oscilação lateral (`wobble`) e rotação inicial e velocidade de rotação.

Retorno: Objeto do sprite ou `null` se o pool estiver vazio.

startRain (`Function ()` → `void`)

Inicializa `fallingPokemons` com 15 sprites pré-posicionados em alturas aleatórias (para não aparecerem todos do topo de uma vez) e chama `animateRain()`.

animateRain (Function () → void)

Loop de animação principal. Em cada frame: 1. Limpa o canvas. 2. Com probabilidade 3% por frame, adiciona um novo sprite (se houver menos de 18). 3. Para cada sprite: atualiza y (queda) e rotation; calcula deslocamento horizontal sinusoidal (wobble) com `performance.now()`; salva contexto, translada, rotaciona, desenha imagem centralizada, restaura contexto. 4. Remove sprites que saíram da tela ($y > \text{height} + \text{size}$). 5. Chama `requestAnimationFrame(animateRain)` para o próximo frame.

11. js/events.js — EVENTOS DE TECLADO E CLIQUE

Registra todos os event listeners de interação do usuário. Recebe as funções de ação como callbacks via parâmetro, desacoplando a lógica de negócio dos eventos.

initEvents (Function (callbacks: Object) → void [exportada])

Registra dois grupos de listeners: **1. keydown global** — comportamento varia por state.currentScreen: • 'title': ↑/↓ navegam menuIndex circularmente; Enter chama onMenuAction. • 'credits': Escape ou Enter chama onCloseCredits. • 'pokedex': se searchInput estiver em foco, ignora teclas. Caso contrário: ↑/↓ navegam listNavIndex na filteredList, chamam onSelectByIndex e onScrollToIndex; Enter confirma seleção; Escape chama onBackMenu. **2. click em .menu-item** — lê data-action do elemento clicado e chama onMenuAction.

onMenuAction

Chamada com 'start' ou 'credits' ao selecionar item de menu.

onCloseCredits

Chamada para fechar a tela de créditos.

onBackMenu

Chamada para voltar ao menu a partir da Pokédex.

onSelectByIndex

Chamada com o índice na filteredList para selecionar Pokémon.

onScrollToIndex

Chamada com o índice para rolar a lista e manter o item visível.

updateMenuSelection

Chamada após mudar menuIndex para atualizar a classe 'selected' nos itens.

12. js/main.js — ORQUESTRADOR PRINCIPAL

Módulo raiz que importa todos os demais e controla o fluxo completo da aplicação: do carregamento inicial até a navegação na Pokédex.

Funções de inicialização e transição de telas

simulateLoading (async Function () → Promise)

Simula uma barra de progresso falsa entre 0% e 100% (incremento aleatório de 1–4% a cada 80ms) e anima o texto de status do Snorlax ('Z', 'Z z', 'Z z z'... em loop a cada 400ms). Resolve a Promise quando o progresso atinge 100%, sinalizando que a 'carga' terminou.

snorlaxWakeUp (async Function () → Promise)

Animação de acordar do Snorlax em etapas com setTimeout: 0ms: muda status para '!' e adiciona classe 'awake' (anima o '!'). 800ms: adiciona 'waking-up' ao container (Snorlax sobe e some). 1400ms: adiciona 'fade-out' à loading-screen (tela some). 2400ms: oculta loading-screen e resolve a Promise.

showTitleScreen (async Function () → Promise)

Remove 'hidden' da titleScreen, define currentScreen='title', aguarda 100ms e adiciona classe 'reveal' ao forestBg (fade-in do fundo). Resolve após 1400ms.

dropLogo (async Function () → Promise)

Adiciona classe 'drop-in' ao logoContainer ativando a animação de queda com bounce. Resolve após 1200ms.

showMenu (Function () → void)

Remove 'hidden' e adiciona 'show' ao menuOptions. Define menuIndex=0 e chama updateMenuSelection() para destacar START inicialmente.

updateMenuSelection (Function () → void)

Percorre todos os .menu-item e adiciona/remove a classe 'selected' com base em se o índice do elemento (i) é igual a state.menuIndex. Controla qual item mostra o cursor ■ e o brilho.

goToPokedex (Function () → void)

Transição da tela de título para a Pokédex: 1. currentScreen='transitioning' (bloqueia eventos durante transição). 2. Adiciona 'fade-out' a titleScreen. 3. Após 800ms: oculta titleScreen, exibe pokedexScreen com animação 'screen-enter', define currentScreen='pokedex' e chama initPokedex().

openCredits (Function () → void)

Define currentScreen='credits', remove 'hidden' e 'fade-out' da creditsScreen e chama initPokemonRain() para iniciar a animação de chuva.

closeCredits (Function () → void)

Adiciona 'fade-out' à creditsScreen, chama stopRain() e após 500ms oculta a tela e retorna currentScreen='title'.

backToMenu (Function () → void)

Faz a transição de saída da Pokédex: adiciona 'screen-exit', aguarda 500ms, oculta a tela, restaura a titleScreen e define currentScreen='title'.

Funções de inicialização da Pokédex

initPokedex (async Function () → void)

Inicializa a Pokédex uma única vez (guarda por state.pokedexInitialized). Sequência: loadBasicPokemonList → inicializa filteredList → renderList → selectPokemon(1) → setupSearch → setupTabs → setupTypeFilter → updateCounters → fetchAllPokemonTypes (em background).

fetchAllPokemonTypes (async Function () → void)

Carrega os tipos de todos os 151 Pokémon em lotes de 20 (para não estourar o limite de conexões simultâneas). Para cada Pokémon sem tipo em pokemonTypeMap, faz fetchPokemon e armazena os tipos. Após cada lote, re-renderiza a lista para que as cores apareçam progressivamente.

Função principal de seleção de Pokémon

selectPokemon (async Function (id: number) → void)

Função central da Pokédex. Executada toda vez que o usuário seleciona um Pokémon. Passos em ordem: 1. Atualiza currentPokemonId e chama highlightSelected(). 2. Adiciona id a seenSet e chama updateCounters(). 3. Faz fetchPokemon(id); se falhar, retorna cedo. 4. Atualiza pokemonTypeMap[id]. 5. Exibe nome (capitalize) e número (#001 formatado). 6. Tenta carregar o GIF animado (gen-V black-white); fallback para artwork oficial se o GIF não existir. Usa transição de opacidade (0→1). 7. Recria os badges de tipo em #pokemon-types e chama updateTypeBackground. 8. Mapeia os 6 stats para IDs de elementos e atualiza largura e cor de cada barra (verde >60%, laranja >35%, vermelho ≤35%). 9. Exibe peso (÷10 kg), altura (÷10 m) e primeira habilidade. 10. Chama updateSaveBtn(), loadEvolutions(id, selectPokemon) e renderList().

id

ID do Pokémon a ser exibido (1–151).

Retorno: void. Atualiza todo o painel esquerdo da Pokédex.

Funções de setup de interações

setupSearch (Function () → void)

Registra listener 'input' no searchInput que chama applyFilters() a cada tecla pressionada.

setupTabs (Function () → void)

Registra listener 'click' em cada .tab. Ao clicar: remove 'active' de todas, adiciona na clicada, atualiza state.currentTab, limpa o searchInput e chama applyFilters().

setupTypeFilter (Function () → void)

Chama updateTypeFilterColor() para colorir o select imediatamente. Registra listener 'change' que atualiza state.currentTypeFilter, atualiza cor e chama applyFilters().

applyFilters (Function () → void)

Aplica todos os filtros ativos sobre state.allPokemonBasic: 1. Aba 'favorites': filtra por state.favorites. 2. Texto de busca: filtra por name.includes(q), número formatado.includes(q) ou id === q. 3. Filtro de tipo: filtra por pokemonTypeMap[p.id].includes(currentTypeFilter). Atualiza state.filteredList, reseta listNavIndex (mas mantém o atual se visível) e chama renderList().

toggleFavorite (Function (id: number) → void)

Adiciona ou remove um ID de state.favorites. Persiste no localStorage ('pokedex-favorites'). Chama updateSaveBtn(), updateCounters() e applyFilters().

selectPokemonByListIndex (Function (index: number) → void)

Wrapper que extrai o ID do item em filteredList[index] e chama selectPokemon(). Usado como callback em initEvents (navegação por teclado).

scrollListToIndex (Function (index: number) → void)

Encontra o item de lista no índice dado e chama scrollIntoView({block:'nearest', behavior:'smooth'}) para mantê-lo visível durante navegação por teclado.

executeMenuAction (Function (a: string) → void)

Despachante de ações do menu: 'start' → goToPokedex(); 'credits' → openCredits().

init (async Function () → void)

Ponto de entrada da aplicação. Executada ao carregar o módulo. Sequência: simulateLoading → snorlaxWakeUp → showTitleScreen → dropLogo → showMenu.

13. css/ — ESTILOS CSS (RESUMO POR ARQUIVO)

Os estilos são divididos em 4 arquivos importados por style.css na ordem correta. A fonte principal é **Press Start 2P** (Google Fonts), que dá o visual retrô de videogame.

css/style.css

Arquivo de entrada. Contém apenas @import dos outros 4 arquivos na ordem: base → screens → components → responsive. Ponto único referenciado no index.html.

css/base.css

Reset global (* margin/padding 0, box-sizing: border-box), estilo do body (background preto, overflow hidden, fonte Press Start 2P), classe .hidden (display: none !important) e keyframe @fadeIn usado em vários módulos.

css/screens.css

Estilos das 4 telas principais: • #loading-screen: posição fixed, z-index 1000, transição de opacidade para fade-out. Animações: breathe (Snorlax sobe/desce), zPulse (texto Z pisca), popExcl (! aparece), wakeUp (Snorlax sai voando). • #title-screen: posição fixed z-500. Inclui black-bar (barra superior preta), forest-bg (fundo floresta com fadeIn ao ganhar .reveal), logo-container (.drop-in dispara logoDrop com bounce), menu-options, bottom-bar (barra vermelha inferior). • #credits-screen: fixed z-600 com fundo blur+escuro, canvas de chuva, efeito de raios girando (lightSpin), grid de member-cards. • #pokedex-screen: fixed z-200 com type-background (2 layers de fundo dinâmico), header vermelho e pokedex-body (flex row).

css/components.css

Componentes reutilizáveis: • Menu: .menu-item com seleção (cursor ■ amarelo piscando via cursorBlink, brilho no .selected). • Painel esquerdo: #panel-left, #pokemon-name-box, #pokemon-sprite-area (com watermark da pokébola), .type-badge, #pokemon-stats (6 .stat-row com .stat-bar animada), #pokemon-info-extra, #evolution-section (.evo-item, .evo-arrow, destaque .current), #save-btn (amarelo/verde). • Painel direito: #panel-right, #search-input (foco vermelho), tabs, #type-filter-select (19 options coloridas individualmente), .poke-list-item (com .poke-list-pokeball, número, nome, dots de tipo, estrela de favorito). • Transições de tela: .screen-enter (screenSlideIn da direita) e .screen-exit (screenSlideOut para esquerda). • .member-card: card de integrante com foto (cover), overlay de gradiente, info absoluta na base (.member-name, .member-social, .member-email, .member-role).

css/responsive.css

Media queries para telas menores: • ≤750px: pokedex-body vira coluna, panels ocupam 100% de largura, tabs ficam horizontais, evo-item menor, stats com fonte menor, credits-members vira 2 colunas. • ≤480px: member-cards ainda menores, fontes reduzidas ao mínimo legível.

DIAGRAMA DE DEPENDÊNCIAS DOS MÓDULOS

Setas indicam quem importa quem. main.js é o único ponto de entrada (raiz).

Módulo	Importa de
main.js	state, dom, api, evolutions, ui, events, rain, utils
api.js	state, config
ui.js	config, state, dom, utils
evolutions.js	state, api, dom, utils
events.js	state, config, dom
rain.js	dom (\$)
dom.js	(nenhum – apenas usa document.getElementById)
config.js	(nenhum – apenas constantes)
state.js	(nenhum – apenas lê localStorage no init)
utils.js	(nenhum – função pura)

Fim da documentação — PokedexModular

Gerado por Claude · maio 2026